Solution: Subarrays with Product Less than a Target

Problem Statement

Given an array with positive numbers and a positive target number, find all of its contiguous subarrays whose **product is less than the target number**.

Example 1:

```
Input: [2, 5, 3, 10], target=30
Output: [2], [5], [2, 5], [3], [5, 3], [10]
Explanation: There are six contiguous subarrays whose product is less
than the target.
```

Example 2:

```
Input: [8, 2, 6, 5], target=50
Output: [8], [2], [8, 2], [6], [2, 6], [5], [6, 5]
Explanation: There are seven contiguous subarrays whose product is less
than the target.
```

Solution

This problem follows the **Sliding Window** and the **Two Pointers** pattern and shares similarities with Triplets with Smaller Sum with two differences:

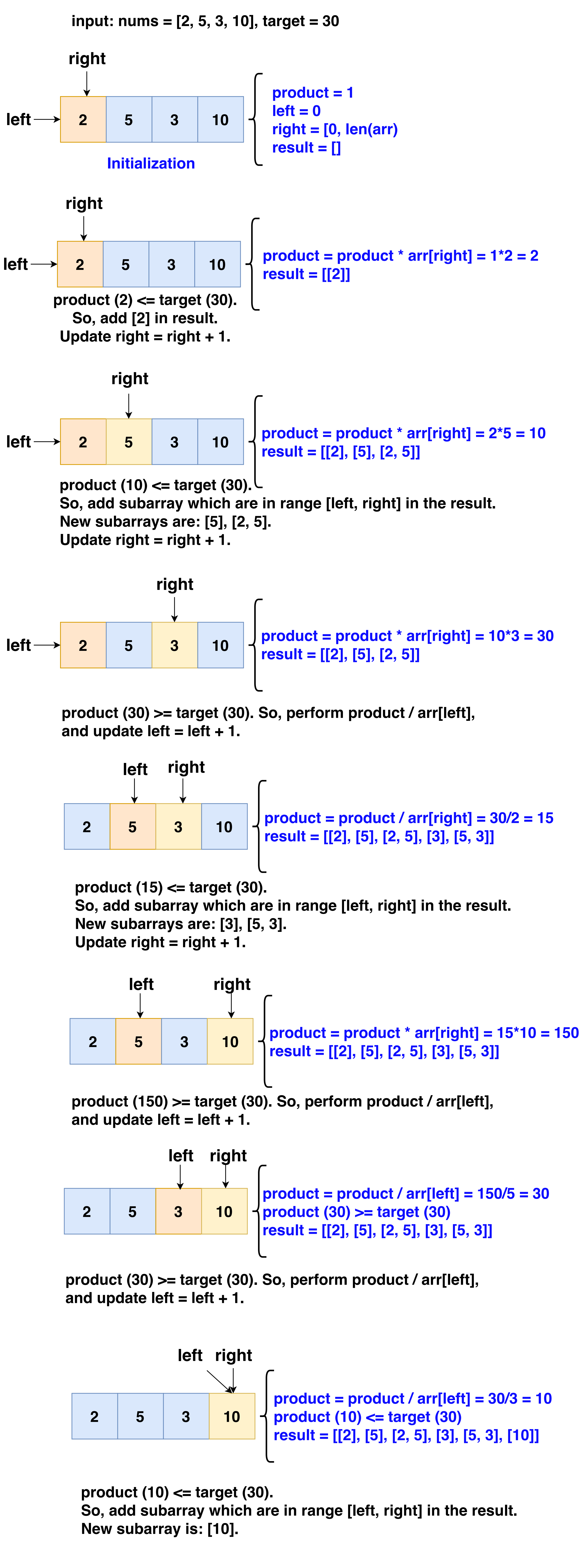
1. In this problem, the input array is not sorted.
2. Instead of finding triplets with sum less than a target, we need to find all subarrays having a product less than the target.

The implementation will be quite similar to Triplets with Smaller Sum. Here is a step-by-step explanation of the algorithm:

1. The goal of this algorithm is to find all subarrays of a given integer array where the product of the numbers in the subarray is less than a specified target value.
2. The algorithm uses a sliding window approach combined with a two-pointer method. A "window" of subarray is defined between the indices pointed to by two pointers, `left` and `right`.
3. The window starts from the leftmost side of the array (both `left` and `right` at position 0) and slides to the right one element at a time, expanding the window. This expansion is represented by incrementing `right`.
4. As we add a new element into the window (the `right` element), we multiply our current `product` with this new element.
5. If at any point the `product` of the numbers within the window (from `left` to `right`) becomes greater than or equal to the `target`, we need to make the product smaller. This is achieved by moving the `left` pointer to the right (incrementing `left`), effectively excluding the `left` element from our window, and dividing our `product` by this removed element. We keep doing this until our `product` is less than `target` again.
6. For each position of `right`, we create all possible subarrays that end at `right`, and that have a product less than the `target`. We do this by starting with a subarray that only includes the `right` element, then progressively adding the element to its left, and so on, until we reach the `left` pointer. All of these subarrays are added to the `result`.
7. This process is repeated for each element in the array (each position of `right`), with the `left` boundary being adjusted as necessary.
8. At the end of this process, `result` will contain all possible subarrays that have a product less than the `target`.

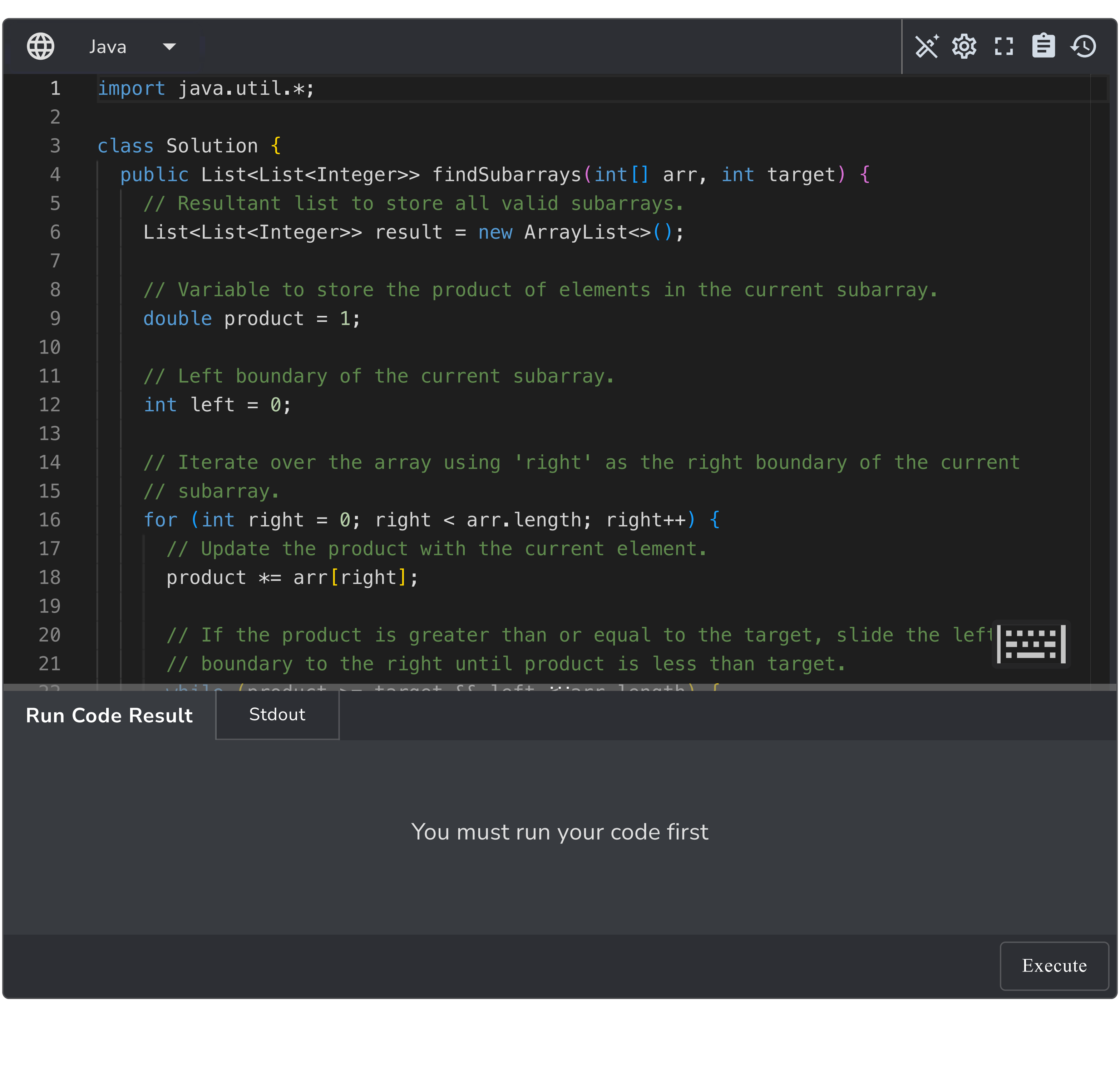
In summary, this algorithm slides a window across the array, expanding and contracting this window as necessary, to find all subarrays where the product of the numbers is less than a target value. For each window, it generates all valid subarrays ending at the right edge of the window.

Let's walkthrough the example 1 through below diagram.


input: nums = [2, 5, 3, 10], target = 30
right
left
2
5
3
10
product = 1
left = 0
right = [0, len(arr)
result = []
Initialization
right
left
product = product * arr[right] = 1*2 = 2
result = [[2]]
product (2) <= target (30).
So, add [2] in result.
Update right = right + 1.
right
left
product = product * arr[right] = 2*5 = 10
result = [[2], [5], [2, 5]]
product (10) <= target (30).
So, add subarray which are in range [left, right] in the result.
New subarrays are: [5], [2, 5].
Update right = right + 1.
right
left
product = product * arr[right] = 10*3 = 30
result = [[2], [5], [2, 5]]
product (30) >= target (30). So, perform product / arr[left],
and update left = left + 1.
left
right
product = product / arr[right] = 30/2 = 15
result = [[2], [5], [2, 5], [3], [5, 3]]
product (15) <= target (30).
So, add subarray which are in range [left, right] in the result.
New subarrays are: [3], [5, 3].
Update right = right + 1.
left
right
product = product * arr[right] = 15*10 = 150
result = [[2], [5], [2, 5], [3], [5, 3]]
product (150) >= target (30). So, perform product / arr[left],
and update left = left + 1.
left
right
product = product / arr[left] = 150/5 = 30
product (30) >= target (30)
result = [[2], [5], [2, 5], [3], [5, 3]]
product (30) >= target (30). So, perform product / arr[left],
and update left = left + 1.
left
right
product = product / arr[left] = 30/3 = 10
product (10) <= target (30)
result = [[2], [5], [2, 5], [3], [5, 3], [10]]
product (10) <= target (30).
So, add subarray which are in range [left, right] in the result.
New subarray is: [10].

Code

Here is what our algorithm will look like:



```java
import java.util.*;

class Solution {
  public List<List<Integer>> findSubarrays(int[] arr, int target) {
    // Resultant list to store all valid subarrays.
    List<List<Integer>> result = new ArrayList<>();

    // Variable to store the product of elements in the current subarray.
    double product = 1;

    // Left boundary of the current subarray.
    int left = 0;

    // Iterate over the array using 'right' as the right boundary of the current
    // subarray.
    for (int right = 0; right < arr.length; right++) {
      // Update the product with the current element.
      product *= arr[right];

      // If the product is greater than or equal to the target, slide the left
      // boundary to the right until product is less than target.
```

You must run your code first

Time Complexity

The main `for-loop` managing the sliding window takes $O(N)$ but creating subarrays can take up to $O(N^2)$ in the worst case. Therefore overall, our algorithm will take $O(N^3)$.

Space Complexity

Ignoring the space required for the output list, the algorithm runs in $O(N)$ space which is used for the temp list.

Can you try estimating how much space will be required for the output list?

```
n + (n-1) + (n-2) + ... 3 + 2 + 1
```

> The worst-case will happen when every subarray has a product less than the target!

> So the question will be, how many contiguous subarrays an array can have?
> It is definitely not all Permutations of the given array; is it all Combinations of the given array?

It is not all the Combinations of all elements of the array!

For an array with distinct elements, finding all of its contiguous subarrays is like finding the number of ways to choose two indices, `i and j`, in the array such that `i <= j`.

If there are a total of n elements in the array, here is how we can count all the contiguous subarrays:

1. When `i = 0, j` can have any value from 0 to n-1, giving a total of n choices.
2. When `i = 1, j` can have any value from 1 to n-1, giving a total of n-1 choices.
3. Similarly, when `i = 2`, `j` can have n-2 choices. ... ...
4. When `i = n-1`, j can only have only 1 choice.

Let’s combine all the choices:

Which gives us a total of: $n * (n+1)/2$.

So, at most, we need space for $O(n^2)$ output lists. At worst, each subarray can take $O(n)$ space, so overall, our algorithm’s space complexity will be $O(n^3)$.

← Previous
Subarrays With Product Less Than A Target (Medium)
Next →
Dutch National Flag Problem (Medium)
☐ Mark as Completed